

Data Manipulation & Preprocessing

Advanced Machine Learning

Sam Ogden

Learning objectives

- Move between NumPy arrays and PyTorch tensors without friction
- Predict tensor shapes through indexing, reshaping, and combining
- Turn a pandas DataFrame into a clean, model-ready tensor
- Spot preprocessing steps that leak information across splits

From raw data to a model

- Every model starts the same way: **raw data** → **tensor** → **model**
- Most of the bugs you'll hit early on are shape bugs, not math bugs
- Today: get fluent moving between numpy, pandas, and torch tensors

NumPy → PyTorch

numpy.ndarray vs torch.Tensor

What's the same

- Both are n-dimensional arrays of numbers
- Same mental model: shape, dtype, indexing, broadcasting
- Mostly the same method names too

What's different

- torch defaults to `float32`, numpy to `float64`
- torch tensors can live on a GPU; numpy arrays can't
- torch can track gradients if you ask it to (`requires_grad`) — more on this in few days

Creating one looks nearly identical

NumPy

```
1 import numpy as np
2
3 a = np.arange(12)
4 print(a)
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
```

PyTorch

```
1 import torch
2
3 t = torch.arange(12)
4 print(t)
```

```
tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

Indexing works the same way

NumPy

```
1 print(a[2:5])
```

```
[2 3 4]
```

PyTorch

```
1 print(t[2:5])
```

```
tensor([2, 3, 4])
```

Reshaping works the same way

NumPy

```
1 print(a.reshape(3, 4))
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

PyTorch

```
1 print(t.reshape(3, 4))
```

```
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

Same values, different default dtype

NumPy

```
1 print(np.ones(3).dtype)
```

float64

PyTorch

```
1 print(torch.ones(3).dtype)
```

torch.float32

- numpy defaults to `float64` — general scientific computing, precision first
- torch defaults to `float32` — half the memory, and GPUs are much faster at it

`np.arange(12)` and `torch.arange(12)` don't give you the same dtype by default. Be explicit when it matters:

```
1 t32 = torch.arange(12, dtype=torch.float32)
2 print(t32.dtype)
```

torch.float32

The explicit connection between the two

NumPy → PyTorch

```
1 shared = torch.from_numpy(a)
2 shared[0] = 99
3 print(a[0])
```

99

PyTorch → NumPy

```
1 back = t.numpy()
2 back[0] = -1
3 print(t[0])
```

tensor(-1)

Both directions share memory — no copy happens. Change one, change the other. Great for speed, easy to get burned by if you're not expecting it.

Foreshadowing: `requires_grad`

```
1 w = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
```

```
2 print(w)
```

```
tensor([1., 2., 3.], requires_grad=True)
```

- `requires_grad=True` tells torch to track every operation done on this tensor
- That tracking is what makes automatic differentiation possible
- This is the thing numpy just doesn't do — we'll build directly on it when we get to training loops

Check your understanding

```
x = torch.arange(6, dtype=torch.float64)
```

Predict:

- `x.dtype`
- `x.numpy().dtype`
- What happens to `x` if you modify `x.numpy()` in place?
- Why did we write `dtype=torch.float64` here instead of just letting it default? When might you actually want `float64`?

Answers:

- `x.dtype` → `torch.float64`
- `x.numpy().dtype` → `float64` — numpy mirrors torch's dtype here
- Modifying `x.numpy()` in place also modifies `x` — they share memory
- `float64` earns its cost when precision loss would actually break something — accumulating many small numbers, ill-conditioned linear algebra, scientific/financial computation. Rare in DL training itself; more common in preprocessing or evaluation code

Shape manipulation

The shape is the *contract*

- Shape encodes what the data *means*: which axis is batch, which is channel, which is a feature
- Most early bugs in this class are shape bugs, not math bugs
- Before you run a line, predict its output shape — that habit is worth more than memorizing the API

Reshape: reinterpreting the same data

```
1 x = torch.arange(1, 10)
2 print(x)
3 print(x.shape)
4
5 square = x.reshape(3, 3)
6 print(square)
7 print(square.shape)
```

```
tensor([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
torch.Size([9])
```

```
tensor([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])
```

```
torch.Size([3, 3])
```

- Same 9 numbers, **same underlying memory**, it's just a different shape way of thinking about the data.
 - Think a 2D array
- Reshape doesn't move or copy data; it changes how the data is *read*

The same idea, at model scale

```
1 x = torch.arange(64 * 1 * 28 * 28).reshape(64, 1, 28, 28)
2 print(x.shape)
3
4 flat = x.reshape(x.shape[0], -1)
5 print(flat.shape)
```

```
torch.Size([64, 1, 28, 28])
```

```
torch.Size([64, 784])
```

- A batch of 64 grayscale 28×28 images, flattened to feed an neural network
- `-1` means “figure this dimension out for me”
 - useful when you don’t want to hardcode it

`cat` joins along an existing axis, `stack` makes a new one

`torch.cat`

```
1 a1 = torch.zeros(2, 3)
2 a2 = torch.ones(2, 3)
3 print(torch.cat([a1, a2], dim=0).shape)
```

`torch.Size([4, 3])`

`torch.stack`

```
1 print(torch.stack([a1, a2], dim=0).shape)
torch.Size([2, 2, 3])
```

- `cat` on two `(2, 3)` tensors along `dim=0` → `(4, 3)` — no new axis, just longer
- `stack` on the same tensors → `(2, 2, 3)` — a brand new axis is created

squeeze / unsqueeze: adding or removing size-1 axes

```
1 example = torch.arange(5)
2 print(example.shape)
3
4 batched = example.unsqueeze(0)
5 print(batched.shape)
```

torch.Size([5])

torch.Size([1, 5])

- A model is built to expect a **batch** of examples, shape `(batch, features)` — even when you only have one example to give it
- `unsqueeze(0)` makes a single example *look like* a batch of one, so it fits the model's expected shape
- `squeeze(0)` undoes it — strip the batch axis back off once you're done

Broadcasting: how PyTorch lines up mismatched shapes

Line the two **shapes** up on their **right** edge. Walk right to left, axis by axis. At each axis, you need either the same number, or one of them to be **1** (which gets stretched to match).

```
shape(batch):  (4, 3)
shape(bias) :   (3, )
               -----
shape(result): (4, 3)
```

- These rows are *shapes*, not the actual numbers in the tensors
- Rightmost axis: **3** vs **3** → match
- Next axis: **4** vs *nothing* → treat the missing axis as **1**, stretch it to **4**

Same rule, in code

```
1 batch = torch.ones(4, 3)
2 bias = torch.tensor([1.0, 2.0, 3.0])
3 print((batch + bias).shape)
```

`torch.Size([4, 3])`

- `bias` (shape `(3, )`) gets added to every one of the 4 rows in `batch`
- No loop, no manual copying — broadcasting does the repeating for you

When broadcasting can't save you

```
shape(batch):  (4, 3)
shape(wrong):   (2, )
-----
shape(result):  x
```

- Again, these rows are *shapes* — `batch` and `wrong` are still just tensors of numbers
- Rightmost axis: `3` vs `2` → neither matches, and neither is `1` → broadcasting gives up here

```
1 batch = torch.ones(4, 3)
2 wrong = torch.tensor([1.0, 2.0])
3 batch + wrong
```

RuntimeError

Traceback (most recent call last)

Cell In[19], line 3

```
1 batch = torch.ones(4, 3)
2 wrong = torch.tensor([1.0, 2.0])
----> 3 batch + wrong
```

RuntimeError: The size of tensor a (3) must match the size of tensor b (2) at non-singleton dimension 1

Predict before you run

```
x = torch.arange(24).reshape(4, 6)
y = x[:, :3]
z = torch.stack([y, y], dim=0)
w = z.reshape(z.shape[0], -1)
```

Predict the shape of **x**, **y**, **z**, and **w** — in that order — before running anything.

```
1 x = torch.arange(24).reshape(4, 6)
2 y = x[:, :3]
3 z = torch.stack([y, y], dim=0)
4 w = z.reshape(z.shape[0], -1)
5 print(x.shape, y.shape, z.shape, w.shape)
```

```
torch.Size([4, 6]) torch.Size([4, 3]) torch.Size([2, 4, 3]) torch.Size([2, 12])
```

Pandas refresher

You've already used this — fast recap

- A DataFrame is a table: rows are examples, columns are features
- Today's version skips the syntax tour and goes straight to what you'll actually reach for

From disk to a DataFrame

```
1 import pandas as pd
2
3 df = pd.DataFrame({
4     "age": [22.0, 35.0, None, 58.0],
5     "fare": [7.25, 71.83, 8.05, 26.00],
6     "sex": ["male", "female", "female", "male"],
7     "survived": [0, 1, 1, 0],
8 })
9 df
```

	age	fare	sex	survived
0	22.0	7.25	male	0
1	35.0	71.83	female	1
2	NaN	8.05	female	1
3	58.0	26.00	male	0

What you check first

Did it parse right?

```
1 df.dtypes
```

```
age          float64
fare         float64
sex           str
survived      int64
dtype: object
```

What's missing?

```
1 df.isna().sum()
```

```
age          1
fare         0
sex           0
survived      0
dtype: int64
```

describe(): a fast sanity check on the numbers

```
1 df.describe()
```

	age	fare	survived
count	3.000000	4.000000	4.000000
mean	38.333333	28.282500	0.500000
std	18.230012	30.294745	0.57735
min	22.000000	7.250000	0.000000
25%	28.500000	7.850000	0.000000
50%	35.000000	17.025000	0.500000
75%	46.500000	37.457500	1.000000
max	58.000000	71.830000	1.000000

- Skim for the obviously wrong: negative ages, a suspicious max, a count that doesn't match the row total

Selecting rows and columns

```
1 df["age"]
```

```
0    22.0
```

```
1    35.0
```

```
2     NaN
```

```
3    58.0
```

```
Name: age, dtype: float64
```

```
1 df.loc[df["sex"] == "female"]
```

	age	fare	sex	survived
1	35.0	71.83	female	1
2	NaN	8.05	female	1

Worth knowing: groupby

```
1 df.groupby("sex")["survived"].mean()
```

```
sex
female    1.0
male       0.0
Name: survived, dtype: float64
```

- A quick sanity check before you build features: does this column even look related to what you're predicting?
- We won't lean on this heavily this semester — just worth knowing it's there

Creating Tensors from DataFrames

Can we just do `torch.tensor(df.values)`?

`df` still has a string column (`sex`) and a missing value (`age`) in it. Let's try converting it straight to a tensor anyway:

```
1 print(df.values.dtype)
```

```
object
```

```
1 torch.tensor(df.values)
```

TypeError

Traceback (most recent call last)

Cell In[29], line 1

----> 1 torch.tensor(df.values)

TypeError: can't convert np.ndarray of type numpy.object_. The only supported types are: float64, float32, float16, complex64, complex32, complex128, complex96, complex192, complex256, complex512, complex1024, complex2048, complex4096, complex8192, complex16384, complex32768, complex65536, complex131072, complex262144, complex524288, complex1048576, complex2097152, complex4194304, complex8388608, complex16777216, complex33554432, complex67108864, complex134217728, complex268435456, complex536870912, complex1073741824, complex2147483648, complex4294967296, complex8589934592, complex17179869184, complex34359738368, complex68719476736, complex137438953472, complex274877906944, complex549755813888, complex1099511627776, complex2199023255552, complex4398046511104, complex8796093022208, complex17592186044416, complex35184372088832, complex70368744177664, complex140737488355328, complex281474976710656, complex562949953421312, complex1125899906842624, complex2251799813685248, complex4503599627370496, complex9007199254740992, complex18014398509481984, complex36028797018963968, complex72057594037927936, complex144115188075855872, complex288230376151711744, complex576460752303423488, complex1152921504606846976, complex2305843009213693952, complex4611686018427387904, complex9223372036854775808, complex18446744073709551616, complex36893488147419103232, complex73786976294838206464, complex147573952589676412928, complex295147905179352825856, complex590295810358705651712, complex1180591620717411303424, complex2361183241434822606848, complex4722366482869645213696, complex9444732965739290427392, complex18889465931478580854784, complex37778931862957161709568, complex75557863725914323419136, complex151115727451828646838272, complex302231454903657293676544, complex604462909807314587353088, complex1208925819614629174706176, complex2417851639229258349412352, complex4835703278458516698824704, complex9671406556917033397649408, complex19342813113834066795298816, complex38685626227668133590597632, complex77371252455336267181195264, complex154742504910672534362390528, complex309485009821345068724781056, complex618970019642690137449562112, complex1237940039285380274899124224, complex2475880078570760549798248448, complex4951760157141521099596496896, complex9903520314283042199192993792, complex19807040628566084398385987584, complex39614081257132168796771975168, complex79228162514264337593543950336, complex158456325028528675187087900672, complex316912650057057350374175801344, complex633825300114114700748351602688, complex1267650600228229401496703205376, complex2535301200456458802993406410752, complex5070602400912917605986812821504, complex10141204801825835211973625643008, complex20282409603651670423947251286016, complex40564819207303340847894502572032, complex81129638414606681695789005144064, complex162259276829213363391578010288128, complex324518553658426726783156020576256, complex649037107316853453566312041152512, complex1298074214633706907132624082305024, complex2596148429267413814265248164610048, complex5192296858534827628530496329220096, complex10384593717069655257060992658440192, complex20769187434139310514121985316880384, complex41538374868278621028243970633760768, complex83076749736557242056487941267521536, complex166153499473114484112975882535043072, complex332306998946228968225951765070086144, complex664613997892457936451903530140172288, complex1329227995784915872903807060280344576, complex2658455991569831745807614120560689152, complex5316911983139663491615228241121378304, complex10633823966279326983230456482242756608, complex21267647932558653966460912964485513216, complex42535295865117307932921825928971026432, complex85070591730234615865843651857942052864, complex170141183460469231731687303715884105728, complex340282366920938463463374607431768211456, complex680564733841876926926749214863536422912, complex1361129467683753853853498429727072845824, complex2722258935367507707706996859454145691648, complex5444517870735015415413993718908291383296, complex10889035741470030830827987437816582766592, complex21778071482940061661655974875633165533184, complex43556142965880123323311949751266331066368, complex87112285931760246646623899502532662132736, complex174224571863520493293247799005065324265472, complex348449143727040986586495598010130648530944, complex696898287454081973172991196020261297061888, complex1393796574908163946345982392040522594123776, complex2787593149816327892691964784081045188247552, complex5575186299632655785383929568162090376495104, complex11150372599265311570767859136324180752990208, complex22300745198530623141535718272648361505980416, complex44601490397061246283071436545296723011960832, complex89202980794122492566142873090593446023921664, complex178405961588244985132285746181186892047843328, complex356811923176489970264571492362373784095686656, complex713623846352979940529142984724747568191373312, complex1427247692705959881058285969449495136382746624, complex2854495385411919762116571938898990272765493248, complex5708990770823839524233143877797980545530986496, complex11417981541647679048466287755595961091061972992, complex22835963083295358096932575511191922182123945984, complex45671926166590716193865151022383844364247891968, complex91343852333181432387730302044767688728495783936, complex182687704666362864775460604089535377456991567872, complex365375409332725729550921208179070754913983135744, complex730750818665451459101842416358141509827966271488, complex1461501637330902918203684832716283019655932542976, complex2923003274661805836407369665432566039311865085952, complex5846006549323611672814739330865132078623730171904, complex11692013098647223345629478661730264157247460343808, complex23384026197294446691258957323460528314494920687616, complex46768052394588893382517914646921056628989841375232, complex93536104789177786765035829293842113257979682750464, complex187072209578355573530071658587684226515959365500928, complex374144419156711147060143317175368453031918731001856, complex748288838313422294120286634350736906063837462003712, complex1496577676626844588240573268701473812127674924007424, complex2993155353253689176481146537402947624255349848014848, complex5986310706507378352962293074805895248510699696029696, complex11972621413014756705924586149611790497021399392059392, complex23945242826029513411849172299223580994042798784118784, complex47890485652059026823698344598447161988085597568237568, complex95780971304118053647396689196894323976171195136475136, complex191561942608236107294793378393788647952342390272950272, complex383123885216472214589586756787577295904684780545900544, complex766247770432944429179173513575154591809369561091801088, complex1532495540865888858358347027150309183618739122183602176, complex3064991081731777716716694054300618367237478244367204352, complex6129982163463555433433388108601236734474956488734408704, complex12259964326927110866866776217202473468949912977468817408, complex24519928653854221733733552434404946937899825954937634816, complex49039857307708443467467104868809893875799651909875269632, complex98079714615416886934934209737619787751599303819750539264, complex196159429230833773869868419475239575503198607639501078528, complex392318858461667547739736838950479151006397215279002157056, complex784637716923335095479473677900958302012794430558004314112, complex1569275433846670190958947355801916604025588861116008628224, complex3138550867693340381917894711603833208051177722232017256448, complex6277101735386680763835789423207666416102355444464034512896, complex12554203470773361527671578846415332832204710888928069025792, complex25108406941546723055343157692830665664409421777856138051584, complex50216813883093446110686315385661331328818843555712276103168, complex100433627766186892221372630771322662657637687111424552206336, complex200867255532373784442745261542645325315275374222849104412672, complex401734511064747568885490523085290650630550748445698208825344, complex803469022129495137770981046170581301261101496891396417650688, complex1606938044258990275541962092341162602522202993782792835301376, complex3213876088517980551083924184682325205044405987565585670602752, complex6427752177035961102167848369364650410088811975131171341205504, complex12855504354071922204335696738729300820177623950262342682411008, complex25711008708143844408671393477458601640355247900524685364822016, complex51422017416287688817342786954917203280710495801049370729644032, complex102844034832575377634685573909834406561420991602098741459288064, complex205688069665150755269371147819668813122841983204197482918576128, complex411376139330301510538742295639337626245683966408394965837152256, complex822752278660603021077484591278675252491367932816789931674304512, complex1645504557321206042154969182557350504982735865633579863348609024, complex3291009114642412084309938365114701009965471731267159726697218048, complex6582018229284824168619876730229402019930943462534319453394436096, complex13164036458569648337239753460458804039861886925068638906788872192, complex26328072917139296674479506920917608079723773850137277813577744384, complex52656145834278593348959013841835216159447547700274555627155488768, complex105312291668557186697918027683670432318895095400549111254310977536, complex210624583337114373395836055367340864637790190801098222508621955072, complex421249166674228746791672110734681729275580381602196445017243910144, complex842498333348457493583344221469363458551160763204392890034487820288, complex1684996666896914987166688442938726917102321526408785780068975640576, complex3369993333793829974333376885877453834204643052817571560137951281152, complex6739986667587659948666753771754907668409286105635143120275902562304, complex13479973335175319897333507543509815336818572211270286240551805124608, complex26959946670350639794667015087019630673637144422540572481103610249216, complex53919893340701279589334030174039261347274288845081144962207220498432, complex107839786681402559178668060348078522694548577690162289924414440996864, complex215679573362805118357336120696157045389097155380324579848828881993728, complex431359146725610236714672241392314090778194310760649159697657763987456, complex862718293451220473429344482784628181556388621521298319395315527974912, complex1725436586902440946858688965569256363112777243042596638790631055949824, complex3450873173804881893717377931138512726225554486085193277581262111899648, complex6901746347609763787434755862277025452451108972170386555162524223799296, complex13803492695219527574869511724554050904902217944340773110325048447598592, complex27606985390439055149739023449108101809804435908681546220650096895197184, complex55213970780878110299478046898216203619608871817363092441300193790394368, complex110427941561756220598956093796432407239217743634726184882600387580788736, complex220855883123512441197912187592864814478435487269452369765200775161577472, complex441711766247024882395824375185729628956870974538904739530401550323154944, complex883423532494049764791648750371459257913741949077809479060803100646309888, complex1766847064988099529583297500742918515827483898155618958121606201292619776, complex3533694129976199059166595001485837031654967796311237916243212402585239552, complex7067388259952398118333190002971674063309935592622475832486424805170479104, complex14134776519904796236666380005943348126619871185244951664972849610340958208, complex28269553039809592473332760011886696253239742370489903329945699220681916416, complex56539106079619184946665520023773392506479484740979806659891398441363832832, complex113078212159238369893331040047546785012958969481959613319782796882727665664, complex226156424318476739786662080095093570025917938963919226639565593765455331328, complex452312848636953479573324160190187140051835877927838453279131187530910662656, complex904625697273906959146648320380374280103671755855676906558262375061821325312, complex1809251394547813918293296640760748560207343511711353813116524750123642650624, complex3618502789095627836586593281521497120414687023422707626233049500247285301248, complex7237005578191255673173186563042994240829374046845415252466099000494570602496, complex14474011156382511346346373126085988481658748093690830504932198000989141204992, complex28948022312765022692692746252171976963317496187381661009864396001978282409984, complex57896044625530045385385492504343953926634992374763322019728792003956564819968, complex115792089251060090770770985008687907853269984749526644039457584007913129639936, complex231584178502120181541541970017375815706539969499053288078915168015826259279872, complex463168357004240363083083940034751631413079938998106576157830336031652518559744, complex926336714008480726166167880069503262826159877996213152315660672063305037119488, complex1852673428016961452332335760139006525652319755992426304631321344126610074238976, complex3705346856033922904664671520278013051304639511984852609262642688253220148477952, complex7410693712067845809329343040556026102609279023969705218525285376506440296955904, complex14821387424135691618658686081112052205218558047939410437050570753012880593911808, complex29642774848271383237317372162224104410437116095878820874101141506025761187823616, complex59285549696542766474634744324448208820874232191757641748202283012051522375647232, complex118571099393085532949269488648896417641748464383515283496404566024103044751294464, complex237142198786171065898538977297792835283496928767030566992809132048206089502588928, complex474284397572342131797077954595585670566993857534061133985618264096412179005177856, complex948568795144684263594155909191171341133987715068122267971236528192824358010355712, complex1897137590289368527188311818382342682267975430136244535942473056385648716020711424, complex3794275180578737054376623636764685364535950860272489071884946112771297432041422848, complex7588550361157474108753247273529370729071901720544978143769892225542594864082845696, complex1517710072231494821750649454705874145814380344108995628753978445108518972816

Step 1: handle missing values

```
1 df["age"] = df["age"].fillna(df["age"].median())  
2 df
```

	age	fare	sex	survived
0	22.0	7.25	male	0
1	35.0	71.83	female	1
2	35.0	8.05	female	1
3	58.0	26.00	male	0

- Decide this deliberately — fill with median? drop the row? A tensor has no concept of “missing,” so this decision has to happen in pandas, before conversion

Step 2: turn categories into numbers

```
1 df["sex_male"] = (df["sex"] == "male").astype(int)
2 df[["age", "fare", "sex_male", "survived"]]
```

	age	fare	sex_male	survived
0	22.0	7.25	1	0
1	35.0	71.83	0	1
2	35.0	8.05	0	1
3	58.0	26.00	1	0

- A tensor is just numbers — no strings allowed
- Here, a simple 0/1 map is enough; with more than two categories you'd reach for `pd.get_dummies`

Step 3: assemble the tensors

```
1 features = df[["age", "fare", "sex_male"]].values
2 labels = df["survived"].values
3
4 X = torch.tensor(features, dtype=torch.float32)
5 y = torch.tensor(labels, dtype=torch.float32)
6
7 print(X.shape, y.shape)
8 print(X)
```

```
torch.Size([4, 3]) torch.Size([4])
tensor([[22.0000,  7.2500,  1.0000],
        [35.0000, 71.8300,  0.0000],
        [35.0000,  8.0500,  0.0000],
        [58.0000, 26.0000,  1.0000]])
```

- X is (examples, features), y is (examples,) — this is the shape every model in this class will expect

The catch: fit preprocessing on train only

```
1 raw = pd.DataFrame({
2     "age": [22.0, 35.0, None, 58.0],
3     "fare": [7.25, 71.83, 8.05, 26.00],
4 })
5
6 train, test = raw.iloc[:3], raw.iloc[3:]
7
8 age_median = train["age"].median()      # fit on train only
9 train = train.fillna({"age": age_median})
10 test = test.fillna({"age": age_median}) # reuse the train statistic
```

Never call `.fillna(raw["age"].median())` on the whole dataset before splitting

- that lets test-set information leak into training.

Any statistic you fit (mean, median, category list, min/max for scaling) gets calculated on train, then reused as-is on validation and test.

Takeaways

- Tensors and numpy arrays are basically the same idea — shape, dtype, indexing, and broadcasting all carry over
- Track shapes deliberately: reshape doesn't move data, `cat/stack/squeeze/unsqueeze` compose predictably, and broadcasting has exactly one rule
- Getting from a DataFrame to a tensor means handling missing values and categoricals explicitly, then fitting any preprocessing statistic on train only

[Backup] Autograd teaser

requires_grad in action

```
1 w = torch.tensor([2.0], requires_grad=True)
2 loss = (w - 5) ** 2
3 loss.backward()
4 print(w.grad)
```

tensor([-6.])

- torch tracked every operation from `w` to `loss`
- `.backward()` computes $d(\text{loss})/d(w)$ automatically — no calculus done by hand
- That gradient is exactly what an optimizer uses to update `w`. This is the entire mechanism training runs on — we'll spend real time here soon